PROJECT KEYSTONE: FIELD SERVICE MANAGEMENT

Comprehensive Technical Architecture, System Implementation & Operations Report

Project Name	KEYSTONE Field Service Platform	**Version**	1.0.0 Production Release
Domain	Enterprise FSM & SLA Compliance	**Backend Stack**	Java 21 / Spring Boot 3.3
Frontend Stack	React 18 / TypeScript / Vite	**Security**	Spring Security & JWT Bearer
Live Web Link	elaborate-pika-073004.netlify.app	**Live Backend API**	keystone-backend-production...

1. Executive Summary

KEYSTONE is a state-of-the-art, full-stack enterprise Field Service Management (FSM) platform crafted to streamline maintenance workflows, dispatch field technicians, monitor real-time Service Level Agreement (SLA) deadlines, and provide transparent self-service capabilities for commercial facility customers.

The platform bridges critical operational gaps between **Facility Managers**, **Operations Dispatchers**, **Field Technicians**, and **Commercial Clients**. By combining dynamic SLA algorithms, interactive 6-stage Kanban board lifecycles, GPS staff tracking, mobile labor/parts logging, and responsive space glassmorphism design, Keystone achieves high operational throughput and reduces mean time to resolution (MTTR).

2. Problem Statement & Key Objectives

2.1 The Operational Challenge

Legacy maintenance operations suffer from serious structural bottlenecks:

- **Unmonitored SLA Breaches:** Inability to track priority resolution deadlines (e.g., EMERGENCY 4h vs LOW 48h), leading to heavy financial penalties.
- **Inefficient Field Dispatching:** Lack of real-time GIS tracking for field technicians, causing delayed responses.
- **Unrecorded Labor & Inventory:** Spare parts consumed on-site and technician hours spent without digital audit trails.
- **Client Opaque Communication:** Facility clients calling support desks repeatedly due to zero visibility into ticket resolution progress.

2.2 Core Project Objectives

- **Automate SLA Compliance Calculation:** Compute live SLA compliance metrics on every work order transition.
- **Enforce State Machine Integrity:** Support strict 6-stage work order transitions (NEW ➔ ASSIGNED ➔ IN_PROGRESS ➔ ON_HOLD ➔ COMPLETED ➔ CLOSED).

- **Implement GIS Staff Tracking:** Prescribe live GPS worker coordinates and provide interactive technician radar.
- **Deliver Premium UI/UX Aesthetics:** Build a responsive glassmorphic dark theme with vibrant neon indicators and fluid animations.

3. Technology Stack & Architecture Specification

Layer	Technology / Framework	Version	Role & Description
Backend Core	Java JDK	21 (LTS)	High-performance robust object-oriented enterprise runtime
Framework	Spring Boot	3.3.1	Microservice REST API container, DI, and bean lifecycle
Security	Spring Security & JWT	Stateless	Bearer token authentication, BCrypt hashing & RBAC filters
Persistence	Spring Data JPA / Hibernate	6.x	ORM entity mappings with EAGER fetch state graphs
Database	H2 / PostgreSQL & Flyway	10.x	Relational ACID database with automated SQL migration scripts
Frontend	React SPA & TypeScript	18.3 / 5.5	Type-safe reactive component tree with interactive state
Build & Bundling	Vite	5.4	Ultra-fast production ESM module bundler
Cloud Hosting	Netlify Cloud	Production	Automated web hosting & continuous SPA deployment

4. Core Platform Modules & Functional Breakdown

4.1 Executive Operations Dashboard

Provides executive management with real-time operational health indicators:

- **Dynamic SLA Compliance Engine:** Calculates live compliance percentage based on completed vs overdue work order target deadlines.
- **SVG Circular Gauge:** Custom SVG progress ring visualizing compliance status (88%+ target).
- **KPI Cards & Critical Breaches Monitor:** Displays active tickets, completed jobs, emergency breaches, and low stock inventory alerts.

4.2 Interactive Kanban Work Order Lifecycle Board

- **6-Stage Status Columns:** Visually manages tickets across NEW, ASSIGNED, IN_PROGRESS, ON_HOLD, COMPLETED, and CLOSED.
- **Dispatch Modal:** Allows dispatchers to assign qualified field technicians to tickets in 1 click.
- **State Machine Validation:** Enforces valid backend workflow transitions.

4.3 Staff Tracking Radar (GIS GPS System)

- **Prescribed Geolocation System:** Automatically prescribes live user location coordinates with GPS location detection.
- **Technician Radar Map:** Visualizes on-duty personnel, current coordinates, and assigned work orders.

4.4 Mobile Field Portal (Technician Workspace)

- **GPS Field Duty Check-In:** One-touch attendance logger updating technician duty status.
- **Time & Parts Logging:** Record labor hours and spare parts used directly on assigned tickets.

4.5 Customer Care Self-Service Portal

- **Service Request Submission:** Commercial facility clients submit maintenance requests with priority levels.
- **Facility Site Management:** View registered sites and track ticket resolution progress live.

4.6 User Profile & Real-Time Action History

- **Credential Management:** Edit name, email, phone, and select avatar photo presets.
- **Audit Trail History:** Real-time log listing all user updates, check-ins, ticket status changes, and parts logs.

5. Database Relational Model & Entity Specifications

Entity	Primary Fields	Relationships & Constraints
User	id, email, password, name, role, phone, lat, lng	Unique email constraint; @OneToMany WorkOrders, TimeLogs
Customer	id, name, contactEmail	@OneToMany Sites
Site	id, name, address, customer_id	@ManyToOne Customer (EAGER); @OneToMany WorkOrders
WorkOrder	id, code, title, priority, status, slaDueAt, site_id, assigned_to_id	@ManyToOne Site (EAGER); @ManyToOne User (EAGER); @OneToMany TimeLogs, PartUsages
Part	id, name, sku, stockQty, unitPrice	Unique SKU; Inventory tracking
PartUsage	id, work_order_id, part_id, quantity	@ManyToOne WorkOrder (EAGER); @ManyToOne Part (EAGER)
TimeLog	id, work_order_id, technician_id, minutesSpent, note	@ManyToOne WorkOrder (EAGER); @ManyToOne User (EAGER)

6. REST API Endpoint Specification

Method	Endpoint URL	Auth Role	Description & Operations
POST	/api/auth/login	Public	Authenticates email/password or OTP; returns JWT bearer token
POST	/api/auth/register	Public	Registers new user account with role assignment
GET	/api/work-orders	Authenticated	Fetches work orders with status/priority filtering and SLA calculations
PUT	/api/work-orders/{id}/status	Tech / Dispatcher	Updates ticket status and recalculates SLA metrics
PUT	/api/work-orders/{id}/assign	Dispatcher / Manager	Dispatches qualified technician to work order
POST	/api/work-orders/{id}/time-log	Technician	Logs labor hours and notes on assigned work order
POST	/api/work-orders/{id}/part-usage	Technician	Logs spare parts consumed and decrements inventory
GET	/api/sites	Authenticated	Returns facility sites for customer ticket submission
GET / PUT	/api/users/profile	Authenticated	Gets or updates logged-in user profile credentials
GET	/api/users/profile/history	Authenticated	Retrieves real-time audit action history for user

7. Security & Self-Healing Authentication Architecture

Keystone implements a multi-layered security architecture designed for both local production backend deployment and seamless cloud static edge preview:

- **Stateless JWT Filter Chain:** Every REST API request is intercepted by JwtTokenFilter to extract and validate the HMAC-SHA512 bearer token.
- **BCrypt Password Hashing:** All user passwords are encrypted using BCrypt with automatic database hash migration on boot.
- **Self-Healing Demo Authentication:** During static web cloud hosting (e.g. Vercel static deployments), the frontend automatically detects static mode, generates valid client-side demo sessions for selected roles, and populates interactive demo datasets.

8. Production Verification & Live Deployment Links

Target Environment	Live URL / Location	Deployment Details
Netlify Live Application	https://elaborate-pika-073004.netlify.app	Automated SPA frontend hosting with Netlify proxy rewrites
Live Cloud Backend	https://keystone-backend-production.onrender.com	Dockerized Java 21 Spring Boot microservice cloud container
GitHub Source Code	https://github.com/Siddhartha836/Keystone-Field-Service-Management	Full-stack Java Spring Boot & React TypeScript repository
Build Configuration	backend/Dockerfile & netlify.toml	Multi-stage Maven Java 21 build & Netlify API proxy routing

9. Conclusion & Submission Sign-Off

Project **KEYSTONE** successfully addresses every requirement of modern Field Service Management. Combining Spring Boot 3 Java enterprise architecture with React 18 TypeScript and a high-end space glassmorphism UI design, Keystone delivers exceptional operational visibility, automated SLA compliance monitoring, and robust field dispatching capabilities.

The project is 100% functional, fully documented, committed to GitHub, and published live on Vercel.